GPU Computing: Overview and Programming Model

Huiyang Zhou

GPU at a Glance



- 1st: Designed for graphics applications

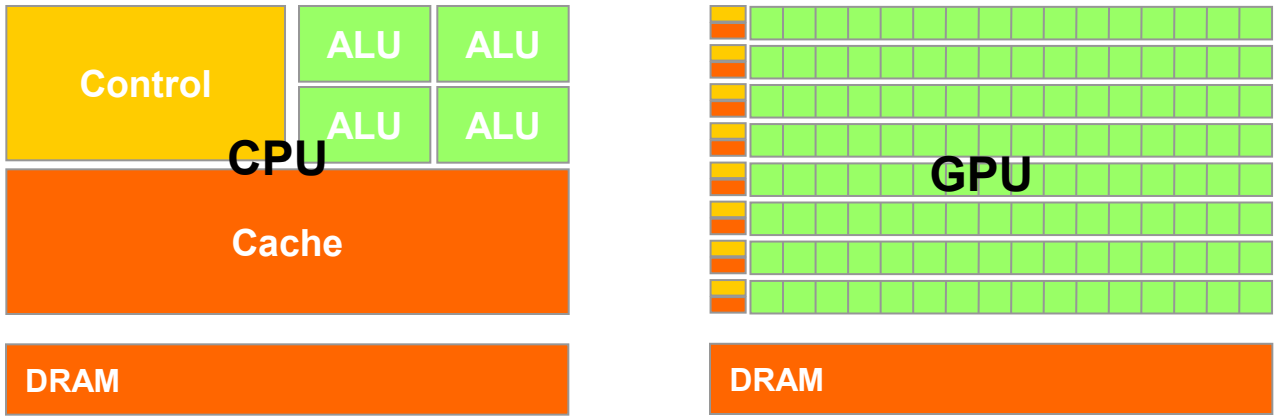
- Trend: converging the different functions into a programmable model

- To suit graphics applications
  - High memory bandwidth
    - >10X CPU memory bandwidth
  - High FP processing power
    - > 10X FLOPS
- Can we utilize the processing power to perform computing besides graphics?
  - GPGPU
  - ML accelerator

GPGPU

GPU vs. CPU

- The GPU is specialized for compute-intensive, highly data parallel computation (exactly what graphics rendering is about)
  - So, more transistors can be devoted to data processing rather than data caching and flow control

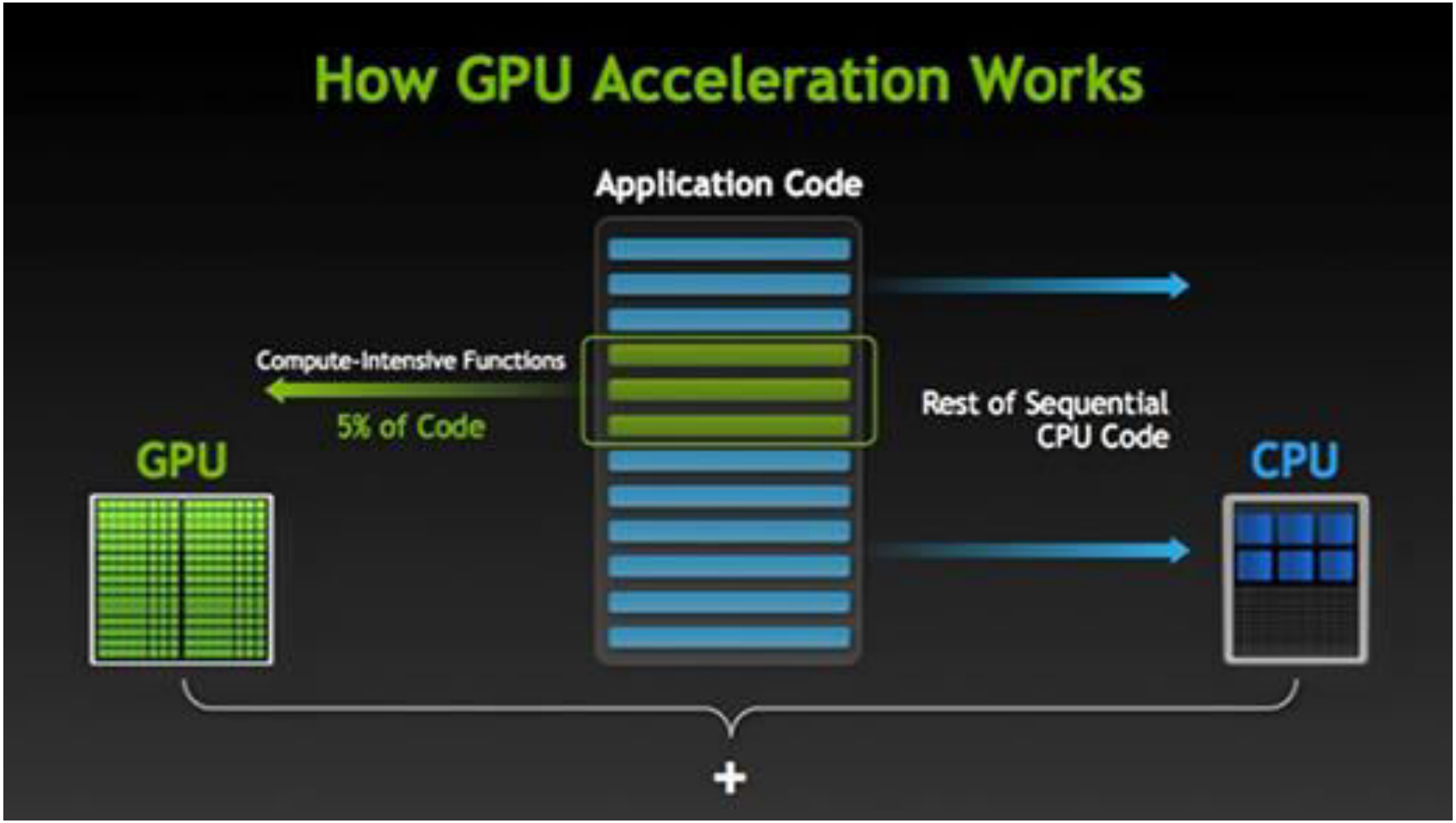


GPU vs. CPU

- CPU: all these on-chip estate are used to achieve performance improvement transparent to software developers
  - Sequential programming model
  - Moving towards multi-core and many-core

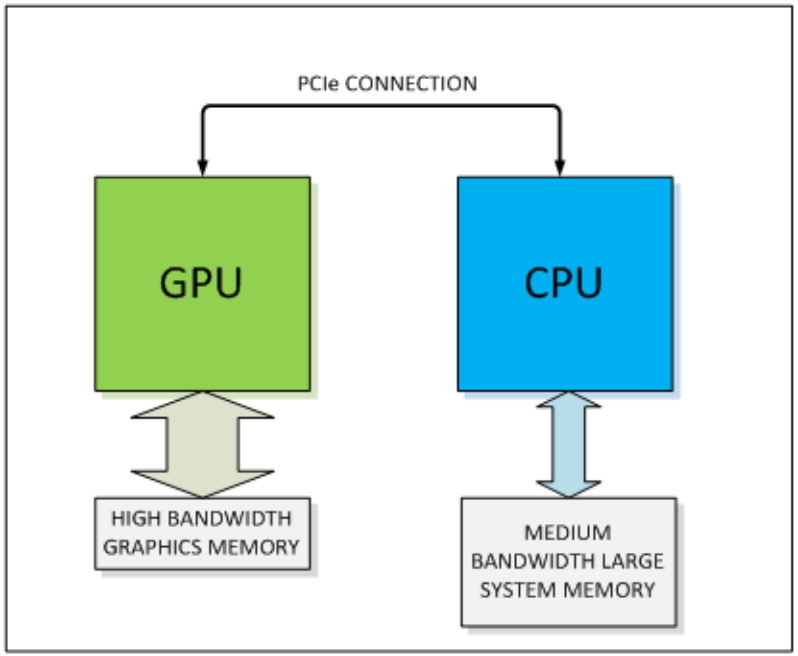
- GPU: more on-chip resources used for floating-point computation
  - Requires data parallel programming model
  - Expose architecture features to software developers and software needs to explicitly take advantage of those features to achieve high performance

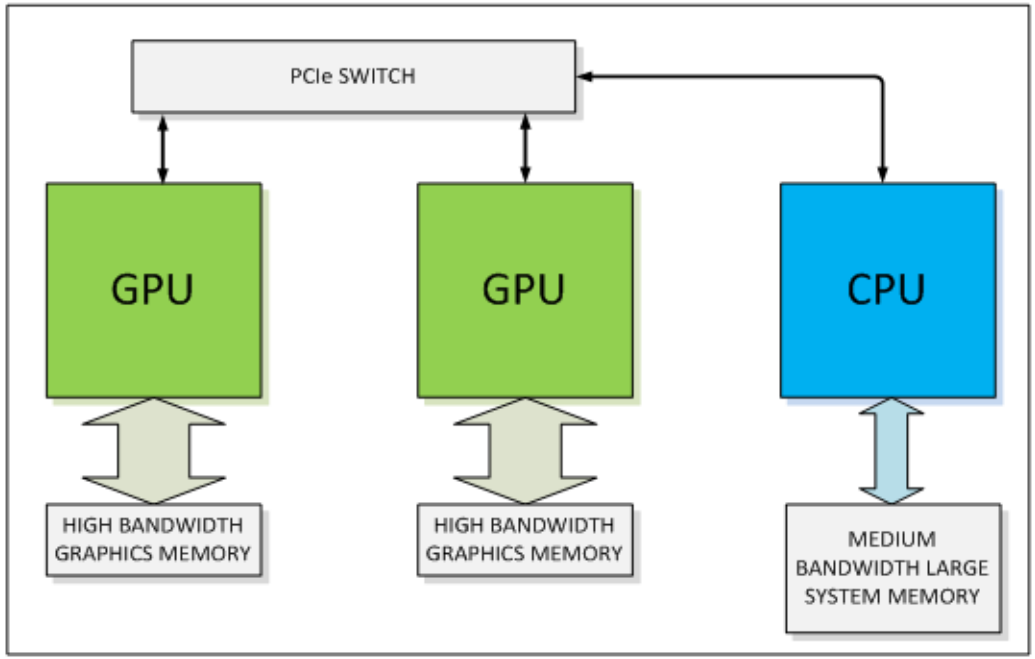
GPU as an Accelerator



http://www.nvidia.com/object/what-is-gpu-computing.html

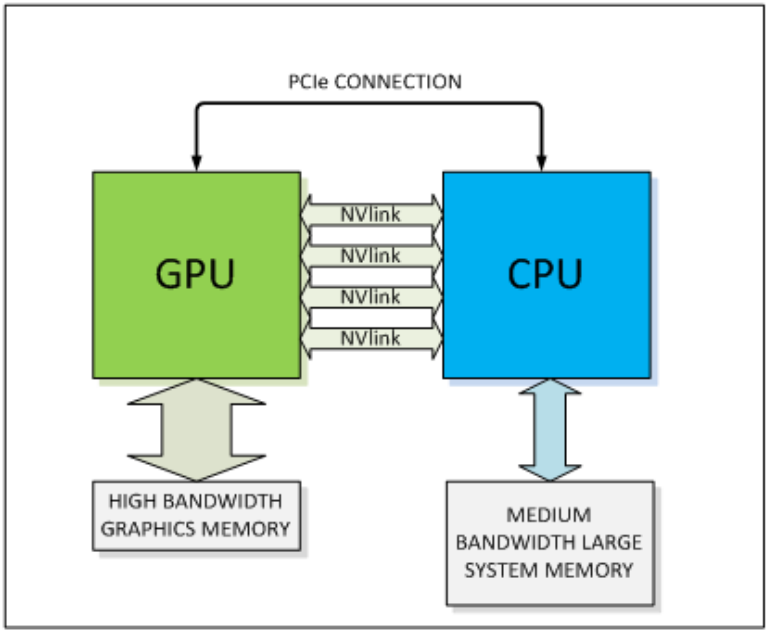
System Architecture

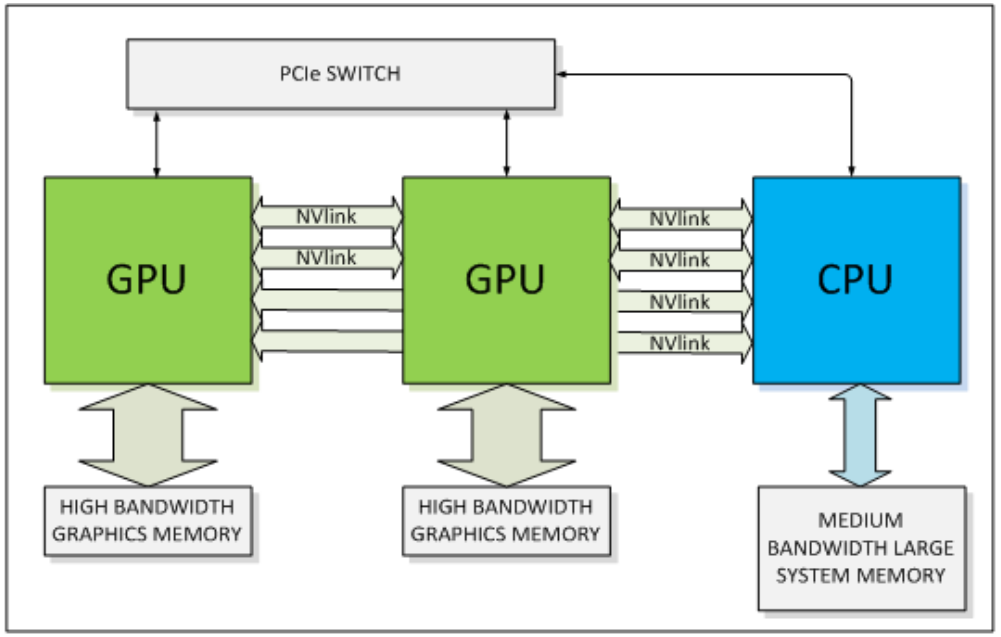




From https://devblogs.nvidia.com/parallelforall/nvlink-pascal-stacked-memory-feeding-appetite-big-data/

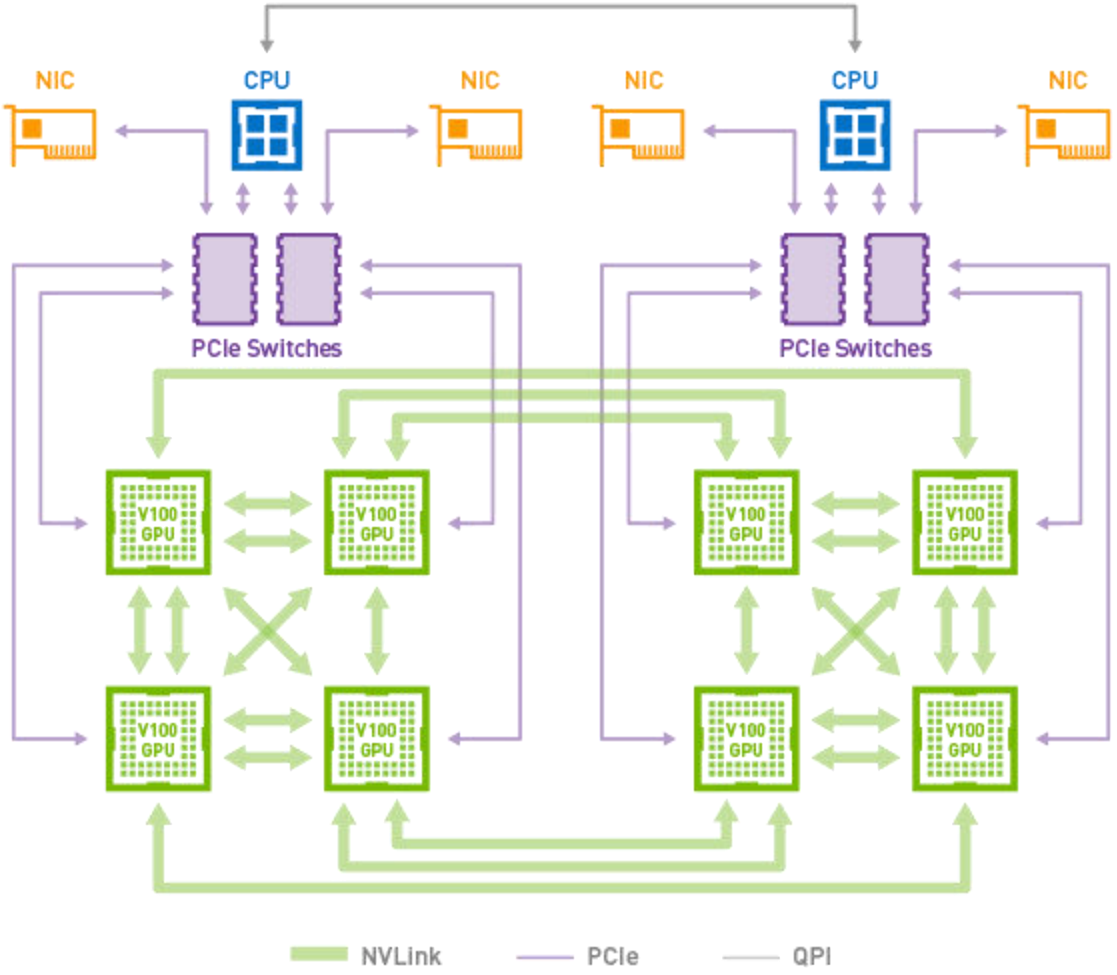
System Architecture





From https://devblogs.nvidia.com/parallelforall/nvlink-pascal-stacked-memory-feeding-appetite-big-data/

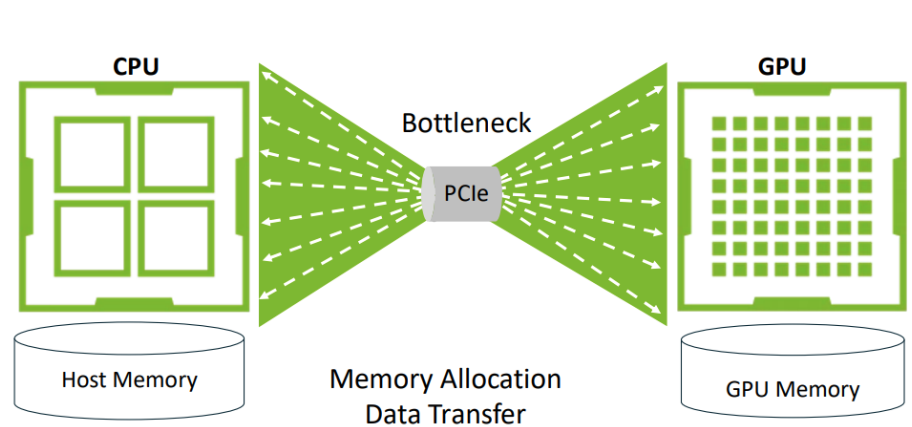
NVIDIA DGX Server

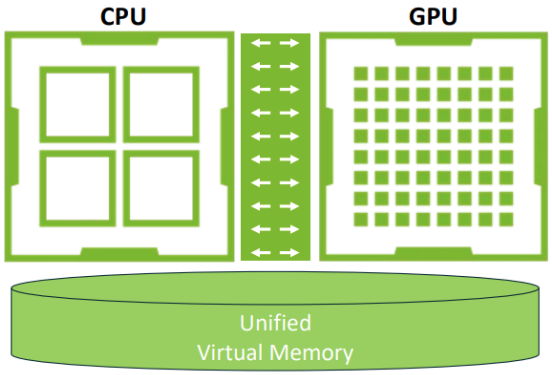


Source: https://www.anandtech.com/show/12587/nvidias-dgx2-sixteen-v100-gpus-30-tb-of-nvme-only-400k

Challenges with Traditional Accelerated Systems

Bridging the GAP: Unifying CPU and GPU Memory





- ❑ PCIe bottlenecks CPU-GPU communication
- ❑ GPU can't access CPU memory directly
- ❑ Complex Programmability
- ❑ Low CPU Performance per Watt

Legacy PCIe Architecture

We need a new chip architecture:

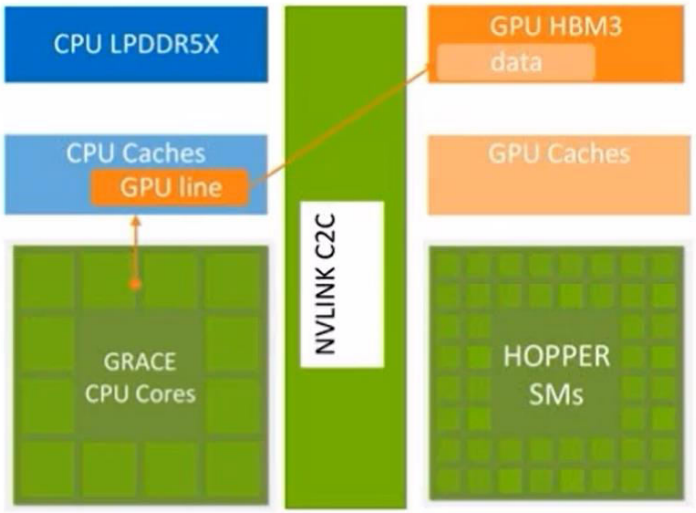
- ✓ Accelerate CPU-GPU data transfers
- ✓ Coherent CPU and GPU cache memory
- ✓ Simplify Programmability
- ✓ Deliver more performance per Watt

NVIDIA

Grace-Hopper Superchip

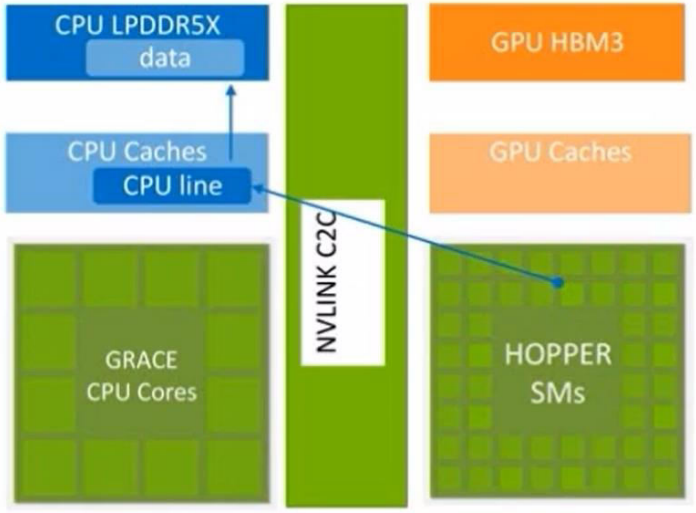
Global Access to All Data

Cache-Coherent Access Via NVLink C2C from Either Processor to Either Physical Memory



Grace directly reading Hopper's memory

CPU fetches GPU data into CPU L3 cache
Cache remains **coherent** with GPU memory
Changes to GPU memory **evict** cache line



Hopper directly reading Grace's memory

GPU loads CPU data via CPU L3 cache
CPU and GPU **can both hit** on cached data
Changes to CPU memory **update** cache line

NVIDIA

Things to know for a GPU processor

- Programming Model
- Instruction Set Architecture
- Thread execution model
  - How the threads are executed, how to synchronize threads
  - How the instructions in each/multiple thread(s) are executed
- Memory model
  - How the memory is organized
  - Speed and Size considerations for different types of memories
  - Shared or private memory. If shared, how to ensure the memory ordering
- Control flow handling

- Support:
  - Programming environment
  - Compiler, debugger, emulator, etc.

CUDA Programming Model: A Highly Multithreaded Coprocessor

- The GPU is viewed as a compute device that:
  - Is a coprocessor to the CPU or host
  - Has its own DRAM (device memory)
  - Runs many threads in parallel
- Data-parallel portions of an application are executed on the device as kernels which run in parallel on many threads
- Differences between GPU and CPU threads
  - GPU threads are extremely lightweight
    - Very little creation overhead
  - GPU needs 1000s of threads for full efficiency
    - Multi-core CPU needs only a few

From the lecture notes of ECE 498 AL by D. Kirk and W. Hwu

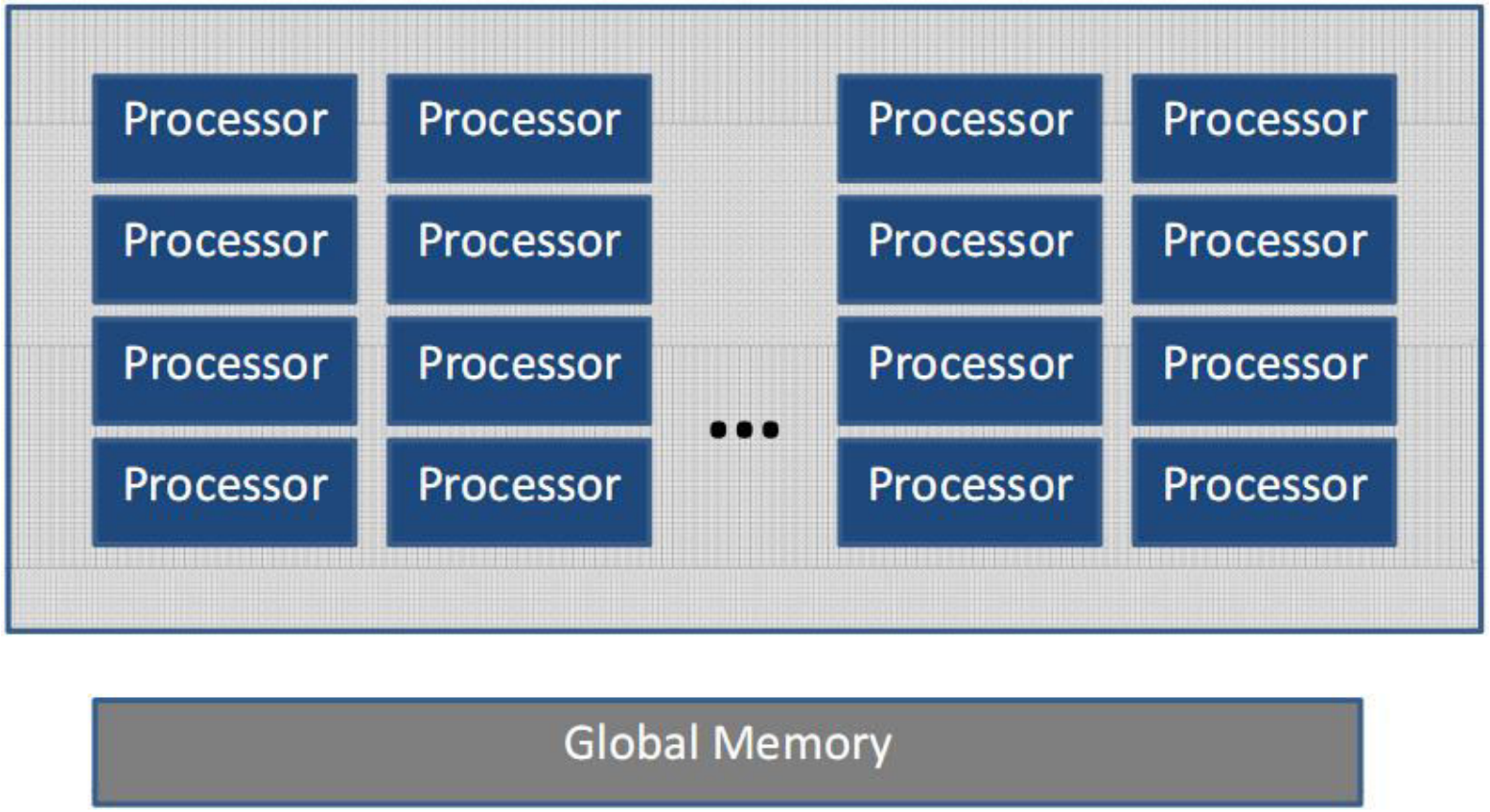
Typical GPGPU Computing Procedure

1. Transfer data from host CPU to GPU memory
2. Invoke GPU kernel function(s)
3. Transfer data back from GPU memory to CPU memory

Unified memory (UM) / Shared virtual memory (SVM) eliminates explicit data movement.

(https://devblogs.nvidia.com/unified-memory-cuda-beginners/)

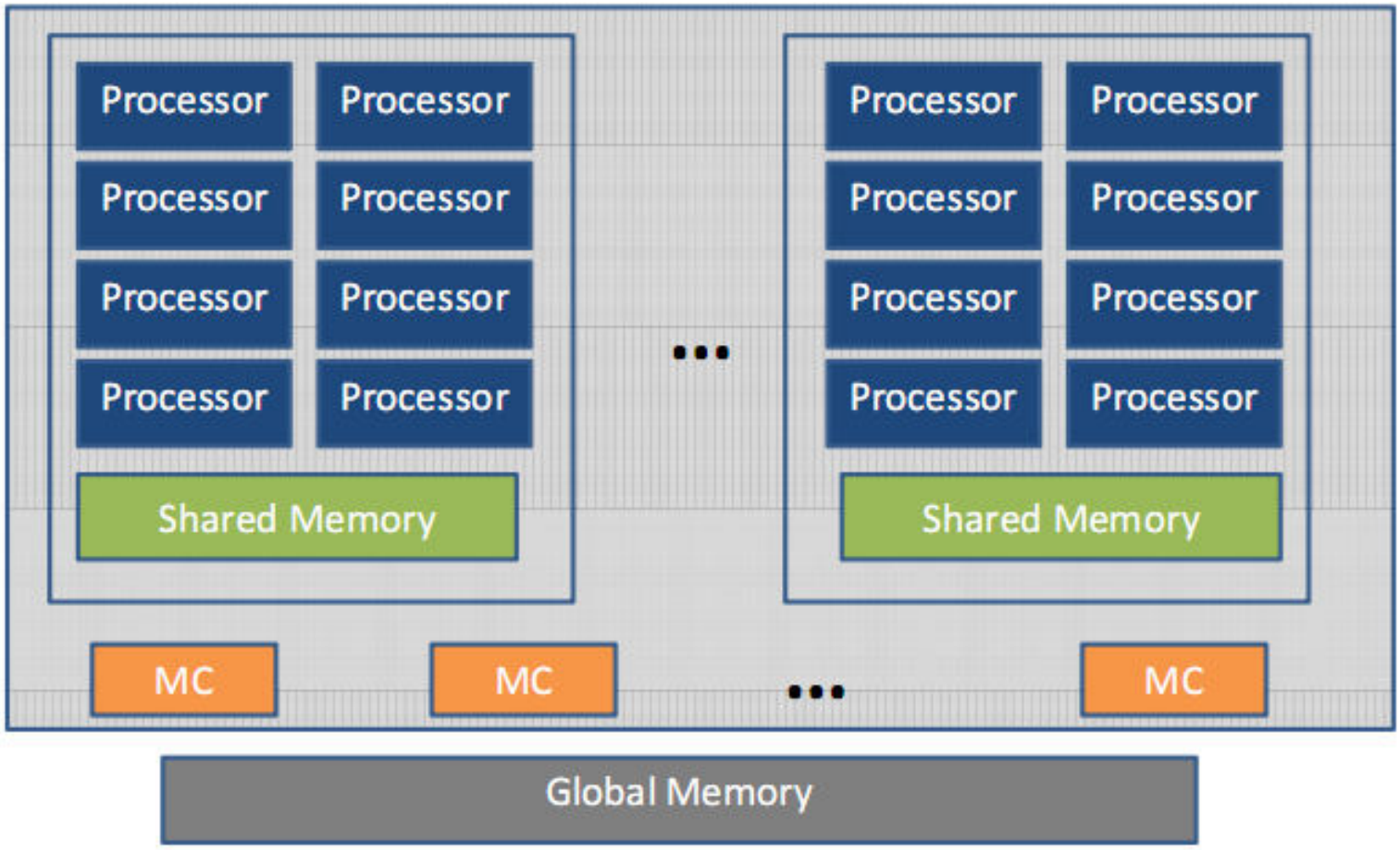
Hardware Abstraction of GPU Architecture



Based on this simple abstraction, develop a naïve implementation without considering optimizations.

Focuses: Data level parallelism; Functional correctness

GPGPU Architecture



- Fast (local) communication among processors in a SM through shared memory
- Memory requests need to be evenly distributed among MCs to avoid conflicts/partition camping

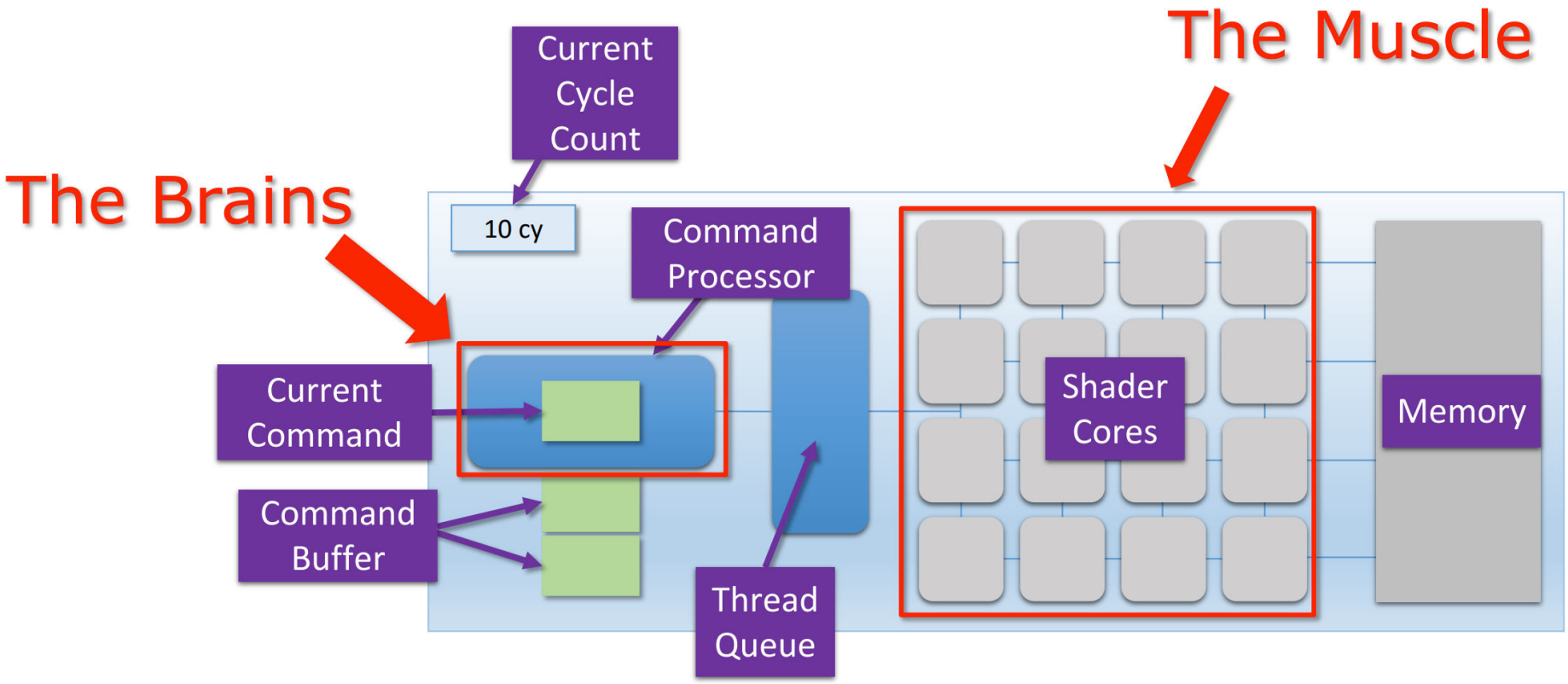
Another Look

GPUs are Thread Monsters!



Src: https://gpuopen.com/gdc-presentations/2019/gdc-2019-agtd5-breaking-down-barriers.pdf

Let's Make a GPU!

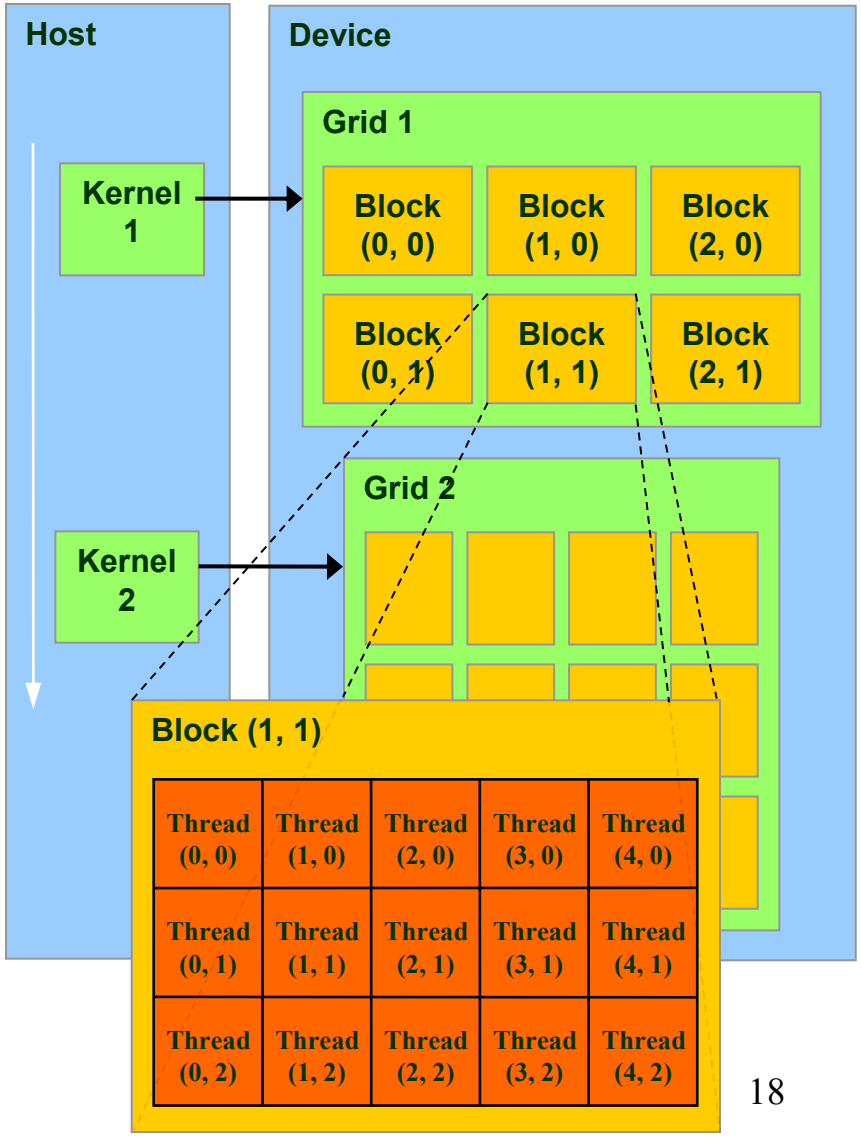


Introducing: The MJP-3000

Src: https://gpuopen.com/gdc-presentations/2019/gdc-2019-agtd5-breaking-down-barriers.pdf

Thread Hierarchy: Grids and Blocks

- A kernel is executed as a grid of thread blocks
  - All threads share data memory space
- A thread block is a batch of threads that can cooperate with each other by:
  - Synchronizing their execution
    - For hazard-free shared memory accesses
  - Efficiently sharing data through a low latency shared memory
- Two threads from two different blocks cannot cooperate

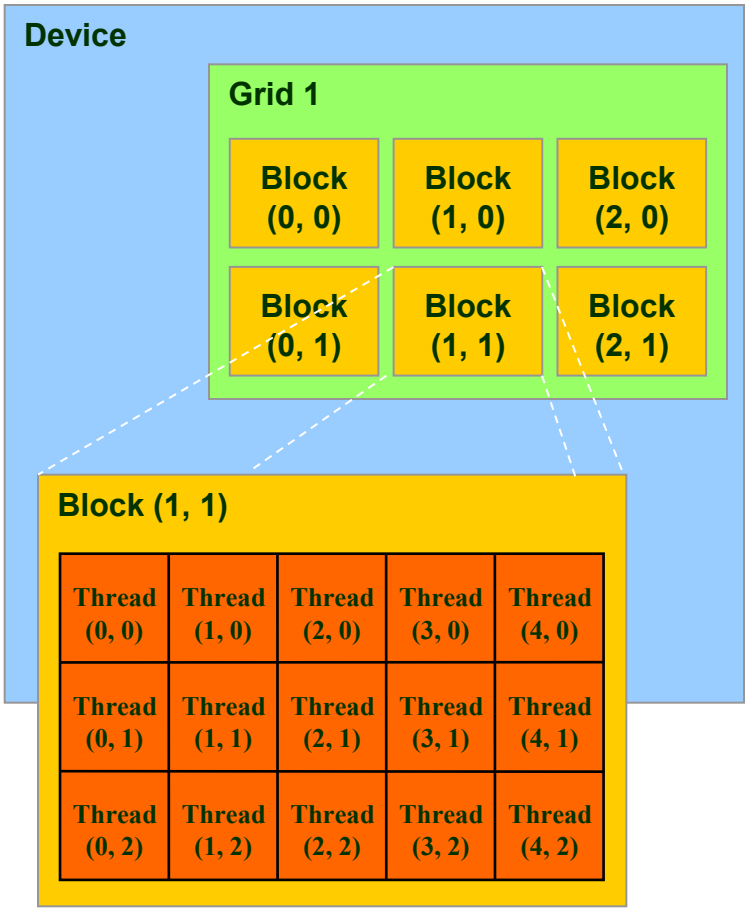


Courtesy: NDVIA

From the lecture notes of ECE 498 AL by D. Kirk and W. Hwu

Block and Thread IDs

- Threads and blocks have IDs
  - So each thread can decide what data to work on
  - Block ID: 1D or 2D
  - Thread ID: 1D, 2D, or 3D
- Simplifies memory addressing when processing multidimensional data
  - Image processing
  - Solving PDEs on volumes
  - …



Courtesy: NDVIA

From the lecture notes of ECE 498 AL by D. Kirk and W. Hwu

Data-Level Parallelism (DLP) as TLP

- How to determine the task of each thread
- Based on algorithm:
  - Each thread handles one or more output elements
  - Each thread handles one or more input elements
  - Element index => thread id

- Based on sequential (CPU) code
  - Loops with high iteration count and NO loop-carried dependence (i.e., one iteration is independent upon others)
  - Loop iterator => thread id
  - Loops with loop-carried dependence
    - Resort back to algorithm.

Example 1: Memory copy

- Memory copy
  - 1D array
  - B[1:N] = A[1:N]

```
__global__ void VecCopy(float* A, float* B) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i < N) B[i] = A[i]; }

int main() {
    ...
    // Kernel invocation with N threads
    VecCopy<<<1, N>>>(A, B);

... }
```

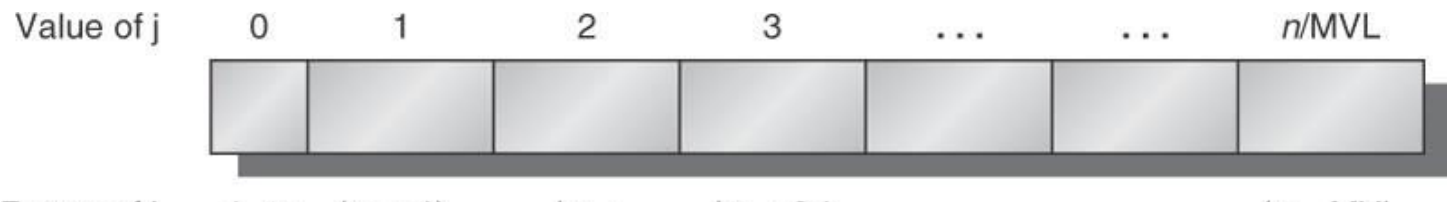
Examples: Vector Copy

```
__global__ void VecCopy(float* A, float* B) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i < N) B[i] = A[i]; }

int main() {
    ...
            // Kernel invocation
    dim3 threadsPerBlock(256,1);
    dim3 numBlocks((N + threadsPerBlock.x – 1) /
                                    threadsPerBlock.x, 1);
    VecCopy<<<numBlocks, threadsPerBlock>>>(A, B);
    …}
```

Vector Copy: Vector Programs

```
low = 1
VL = (n mod MVL) // MVL: Maximal Vector Length
for ( j = 0, j < (n / MVL); j++) { //VL: Vector Length
    B[low: low+VL] = A[low: low + VL]  //Vector Op
low = low + VL /*start of next vector*/
VL = MVL /*reset the length to max*/
}
```



Examples: Matrix Copy

- Memory copy
  - 2D array

```
__global__ void VecCopy(float*A, float* B) {
      int i = blockIdx.x * blockDim.x + threadIdx.x;
      int j = blockIdx.y * blockDim.y + threadIdx.y
      B[j*N+i] = A[j*N+i]; }
int main() {
      ...
      // Kernel invocation
      dim3 threadsPerBlock(16, 16);
      dim3 numBlocks(N / threadsPerBlock.x, N /
                                          threadsPerBlock.y);
      VecCopy<<<numBlocks, threadsPerBlock>>>(A, B);
... }
```

Vectorizing Matrix Copy

Scalar code

```
for(i = 0; i < N; i++)
   for(j = 0; j < N ; j++)
      B[i][j] = A[i][j];
```



Vector-like code: multiple 1D vector memory copy

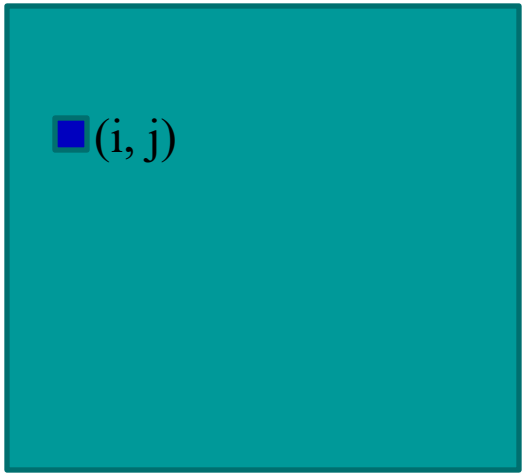
```
for(i = 0; i < N; i++)
   B[i][0:N-1] = A[i][0:N-1];
```

```
for(j = 0; j <N; j++)
   B[0:N-1][j] = A[0:N-1][j]
```

Example 2: Matrix Multiplication

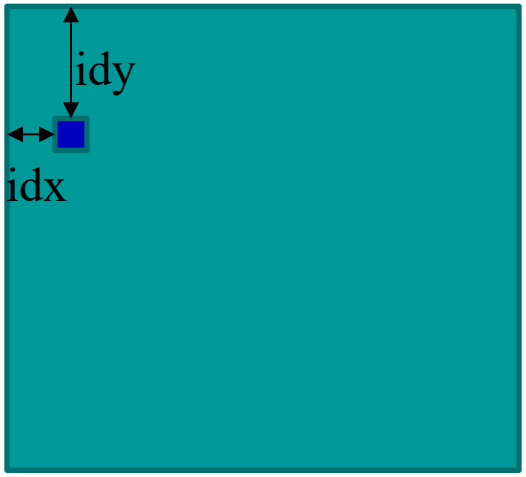
```
for( i = 0; i < N; i++)
    for(j = 0; j < N; j++)
    {
        C[i][j] = 0;
        for(k = 0; k < N; k++)
            C[i][j] += A[i][k] * B[k][j];
    }
```

C[i][j]: (i^{th} row, j^{th} column)



Matrix Multiplication: GPU Code

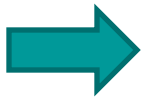
```
__global__ void MatMul(float*A, float* B, float * C, int N) {
      int j = blockIdx.x * blockDim.x + threadIdx.x; //idx
      int i = blockIdx.y * blockDim.y + threadIdx.y; //idy
      sum = 0;
      for(k = 0; k < N, k++)
            sum += A[i*N+k] * B[k*N+j];
     C[i*N + j] = sum;
      }
int main() {
      ...
      // Kernel invocation
      dim3 threadsPerBlock(16, 16);
      dim3 numBlocks(N / threadsPerBlock.x, N /
                                          threadsPerBlock.y);
      MatMul<<<numBlocks, threadsPerBlock>>>(A, B, C, N);
... }
```



Vectorizing Matrix Multiplication

Scalar code

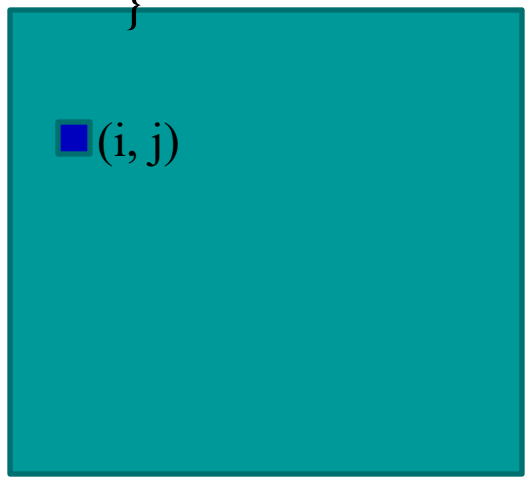
```
for( i = 0; i < N; i++)
    for(j = 0; j < N; j++)
    {
        C[i][j] = 0;
        for(k = 0; k < N; k++)
            C[i][j] += A[i][k] * B[k][j];
    }
```



Vector-like code

```
for( i = 0; i < N; i++)
    for(k = 0; k < N; k++)
    {
        C[i][0:N-1] = 0;
        C[i][0:N-1] += A[i][k] *
    B[k][0:N-1];
    }
```

C[i][j]: (i^{th} row, j^{th} column)



Example

- https://maharshi.bearblog.dev/optimizing-softmax-cuda/

Example 3: Reduction

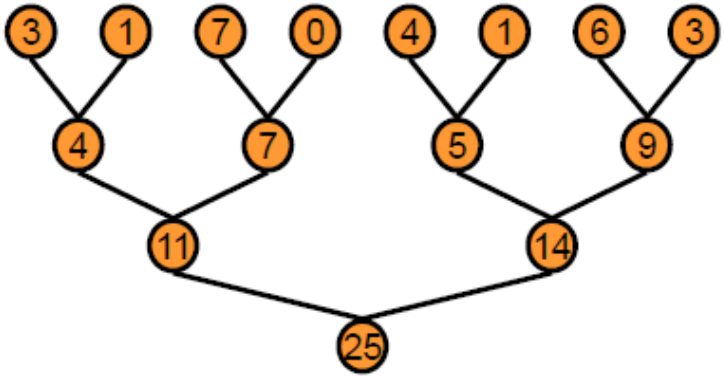
```
sum = 0;
for(i = 0; i < N; i++)
      sum += A[i];
```

Loop-carried dependence

Parallel Reduction

- One thread block processes a vector

- **Tree-based approach used within each thread block**



GPU Reduction Code (un-optimized)

```
__global__ void reduce0(int *g_idata, int *g_odata){
    extern __shared__ int sdata[];
    // each thread loads one element from global to shared mem
   unsigned int tid = threadIdx.x;
   unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
   sdata[tid] = g_idata[i];
   __syncthreads();
   // do reduction in shared mem
   for(unsigned int s=1; s < blockDim.x; s *= 2) {
        if (tid % (2*s) == 0) {sdata[tid] += sdata[tid + s]; }
        __syncthreads(); }
   // write result for this block to global mem
   if (tid == 0)    g_odata[blockIdx.x] = sdata[0];
}
```

CUDA with Cooperative Group

- Since CUDA 9.0, more flexible synchronization among threads, including intra-thread-block and across-thread-blocks, has been introduced.

- https://www.youtube.com/watch?v=rOUaco3w7mU

Vector Reductions

Problem: Loop-carried dependence on reduction variables

```
sum = 0;
for (i=0; i<N; i++)
    sum += A[i];  # Loop-carried dependence on sum
```

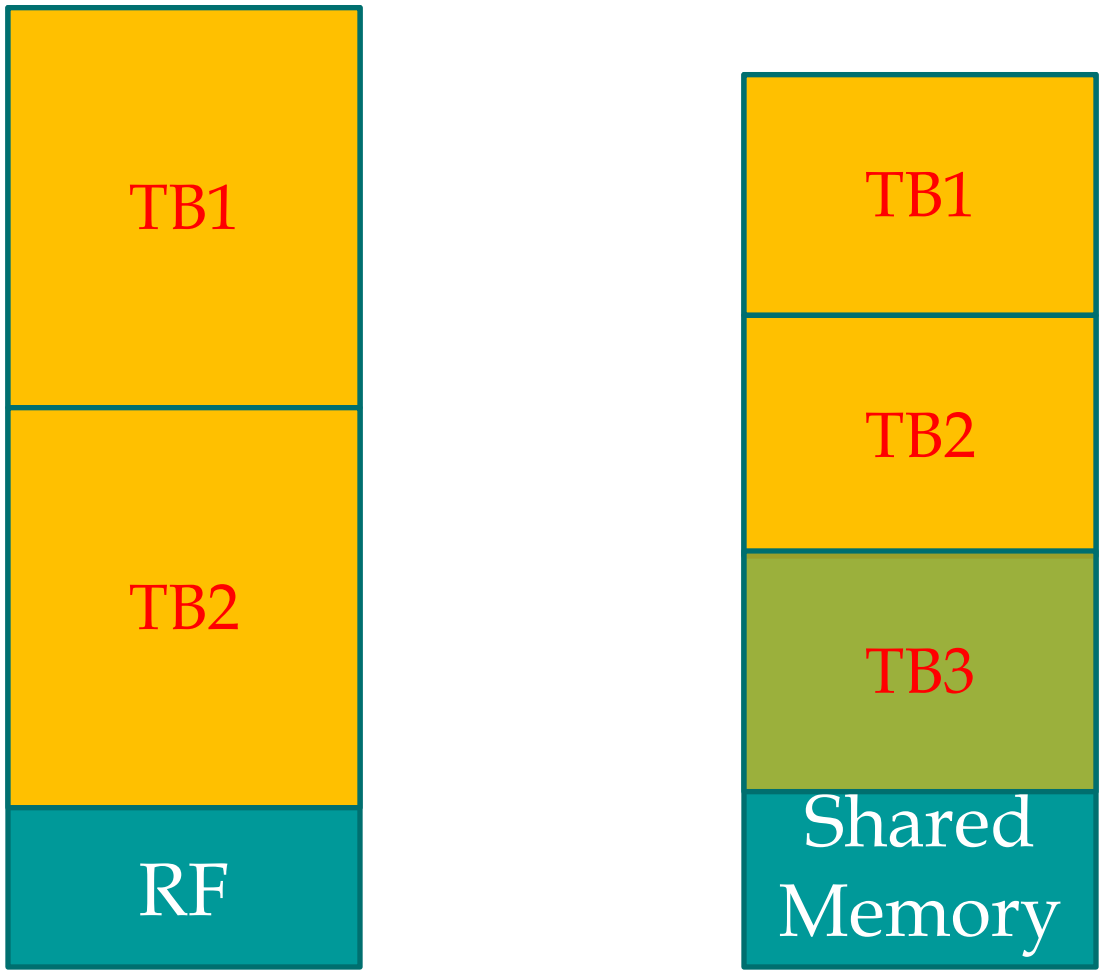
Solution: Re-associate operations if possible, use binary tree to perform reduction

```
# Rearrange as:
sum[0:VL-1] = 0                     # Vector of VL partial sums
for(i=0; i<N; i+=VL)                # Stripmine VL-sized chunks
    sum[0:VL-1] += A[i:i+VL-1]; # Vector sum
# Now have VL partial sums in one vector register
do {
    VL = VL/2;                          # Halve vector length
    sum[0:VL-1] += sum[VL:2*VL-1] # Halve no. of partials
} while (VL>1)
```

Thread-Level Parallelism & Occupancy

- (Effective) TLP: The number of concurrently running threads on each SM, aka occupancy.
- It is limited by resources available on each SM and the resource requirement of each thread block.
  - Maximal # of threads (or warps)
  - Maximal # of thread blocks
  - Register usage of each thread block
    - The number of concurrent thread blocks in an SM <= register file size / register usage of a thread block
  - Shared memory usage of each thread block
    - The number of concurrent thread blocks in an SM <= shared memory size / shared memory usage of a thread block

Example



Persistent Kernels

- A kernel that stays resident on the GPU and processes many tasks in a loop.
- Threads or warps repeatedly fetch work from a queue.

- **Why Persistent kernels?**
  - Reduce kernel (CTA or TB) launch overhead.
  - Better handling of unknown input size
  - Better load balancing for uneven workloads.
  - Keep data in registers/shared memory across tasks.
  - Common in RL, MoE, attention, and streaming workloads.

Traditional vs Persistent Kernel Model

- **Traditional kernel**
  - Launch → execute → exit
  - Grid-wide work partitioned statically

```
//GPU Code
__global__ void traditional(float* x) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) x[i] *= 2.0f;
}
//CPU code
traditional<<<grid, block>>>(...);  // gridDim.x = total_work/work_per_thread

Or
for(…)  //handle both small and large input sizes
 traditional<<<grid, block>>>(...);
```

Persistent Kernel Example

- **Persistent kernel**
  - Launch once
  - Threads stay resident
  - Dynamically pull work from queues

```
__device__ int global_work_idx = 0;
__global__ void persistent(float* x, int N) {
    while (true) {
        int idx = atomicAdd(&global_work_idx, 1);
        if (idx >= N) break;
        x[idx] *= 2.0f;
}}

//CPU code
// Choose a *small* grid for persistence
 int threads_per_block = 256;

// 1–2 blocks per SM
 int num_sms;
 cudaDeviceGetAttribute(&num_sms, cudaDevAttrMultiProcessorCount, 0);
  int blocks = num_sms * 2;
    // Launch ONCE
    persistent<<<blocks, threads_per_block>>>(…);
```

Warp Specialization

- Warp specialization assigns roles per warp, not per thread
- Enables **producer–consumer pipelines inside a thread block**
- Reduces **memory traffic**, **latency**, and **kernel launch overhead**
- Common in:
  - Persistent kernels
  - Fused ML operators

Warp Specialization Example (buggy)

```
__global__ void persistent_warp_specialized(float* in, float* out, int N) {
   __shared__ float buffer[256];


   int warp = threadIdx.x / warpSize;
   int lane = threadIdx.x % warpSize;


   while (true) {
      int idx;
      if (warp == 0 && lane == 0) {
        idx = atomicAdd(&global_work_idx, 1);
      }
      idx = __shfl_sync(0xffffffff, idx, 0);
      if (idx >= N) break;

      if (warp == 0) buffer[lane] = in[idx * warpSize + lane];
      if (warp == 1) buffer[lane] *= 2.0f;
      if (warp == 2) out[idx * warpSize + lane] = buffer[lane];
   }
}
```

Correct Version

```
__device__ int global_work_idx = 0;

__global__ void persistent_warp_specialized(float* in, float* out, int N) {
  __shared__ float buffer[256]; // per-CTA pipeline buffer
  __shared__ int idx_shared; // shared work index

  int warp = threadIdx.x / warpSize;
  int lane = threadIdx.x % warpSize;
  while (true) {
    // One thread acquires work for the whole CTA
    if (warp == 0 && lane == 0) {
      idx_shared = atomicAdd(&global_work_idx, 1);}

    // Ensure all warps see idx_shared
    __syncthreads();

    int idx = idx_shared;
    if (idx >= N) break;

    // Stage 1: load (warp 0)
    if (warp == 0) {buffer[lane] = in[idx * warpSize + lane];}
    __syncthreads(); // ensure load completes

    // Stage 2: compute (warp 1)
    if (warp == 1) {buffer[lane] *= 2.0f;}
    __syncthreads(); // ensure compute completes

    // Stage 3: store (warp 2)
    if (warp == 2) {out[idx * warpSize + lane] = buffer[lane];}
    __syncthreads(); // keep warps in lockstep for next iteration
  }
}
```